

AI Support Ticket Triage

Full-Stack Developer / Software Engineer · Skygnosis

EFFORT

~1 day

DEADLINE

48 hours from receipt

STACK

React/Next · Node/Python · DB · LLM

AI TOOLS

Allowed & encouraged

This task mirrors what we do every day — take a raw input, add an AI layer that creates real value, and ship a clean, working product fast. It touches every layer of the stack and shows how you make trade-offs under a deadline. We care more about clear thinking and quality than about ticking every box.

1 The Brief

Build an app where users submit support tickets and AI automatically triages each one. An admin can then review the triaged tickets on a dashboard.

User side

- A form to submit a ticket: **name**, **email**, and **message**.
- On submit, call an LLM API to automatically generate a **priority** (Low / Medium / High), a **category** (e.g. Billing, Bug, Feature Request, Other), and a **one-line suggested reply**.
- Store all of this and show a confirmation to the user.

Admin side

- A protected dashboard (basic login/auth) that lists all tickets.
- Display the AI-generated priority, category, and suggested reply for each ticket.
- Allow **filtering by priority and category**, and sorting by newest first.

2 Technical Requirements

- **Frontend:** React or Next.js — clean and responsive. No UI library required, but it should look tidy.
- **Backend:** Node/Express or Python (FastAPI/Django) — a REST API with proper endpoints and status codes.
- **Database:** PostgreSQL or MongoDB — a sensible schema.
- **AI:** any LLM (Gemini / OpenAI / Groq free tier). **Handle the failure case gracefully** — what happens if the AI call errors or times out?
- **Code quality:** clear structure, separation of concerns, meaningful commit history.

Note — You do **not** need to deploy the app anywhere. We only need the source code and a video of it running (see below).

What to Submit

- **Frontend GitHub repository** — public link (or invite us as collaborators).
- **Backend GitHub repository** — public link (separate repo, or a clearly structured monorepo is fine).
- **A screen-recording video** (3–7 min) showing the project **running locally** end-to-end: submit a ticket, show the AI triage result, then log into the admin dashboard and demonstrate filtering. Briefly talk through one code decision you are proud of. A Loom link or an uploaded MP4 / Drive link is fine.
- **A short README** in each repo — how to run it, your tech choices and *why*, one trade-off you made because of the deadline, and what you would improve with more time.

3 Ground Rules

- **Timebox** Around one day of effort; 48 hours to submit. Quality of thinking beats completing every feature.
- **AI tools** Cursor, Copilot, Replit and similar are encouraged — we want to see how you use them.
- **Ownership** Be ready to explain any part of your code in a follow-up call. The video walkthrough helps us see it is genuinely yours.
- **Questions** If anything is unclear, make a reasonable assumption, note it in your README, and keep moving.

4 How We Score It (out of 100)

Area	Pts	What we look for
Functionality (works end-to-end)	20	Full flow runs in the video: submit → AI triage → admin review
Architecture & code quality	20	Structure, separation of concerns, readability
API & database design	15	Clean endpoints, sensible schema, error handling
AI integration	15	Correct, robust, gracefully handles failures
UI / UX	10	Clear, responsive, genuinely usable
Git hygiene & video walkthrough	10	Meaningful commits; clear demo of it running
README & product thinking	10	Clear reasoning and honest trade-offs

Area	Pts	What we look for
Bonus	+10	Strong auth, tests, search, or a smart no-code / automation tool used where it made sense
Total	100	+10 bonus available

A tip from us: we would rather see a smaller app that is clean, works, and is well-explained than a large one that is half-broken. Show us how you think, not just what you can build. Good luck — we are looking forward to seeing it.